

Phase 3 학습 계획 — Qwen3-14B QLoRA

TransferJudge · RunPod GPU 환경 학습 가이드

2026.05.14 · 빅데이터학과 17기 곽민아

한 줄 요약

Phase 2에서 생성한 578줄 학습 데이터로 Qwen3-14B 모델을 QLoRA 방식으로 파인튜닝. RunPod A100 80GB 1대, 약 6~8시간, \$3~\$5 예상. 산출물은 약 50MB LoRA adapter.

0. 핵심 지표

578 학습 데이터 (lines)	14B 모델 파라미터	~0.3% 실제 학습 (LoRA)	~7h 예상 시간
\$3~\$5 예상 비용			

1. Phase 3의 목적과 위치

측면	내용
목적	Phase 2에서 GPT-4o-mini가 만든 시범 답안 578개를 Qwen3-14B Judge 모델이 따라 학습 하도록 함
방식	지식 증류(Knowledge Distillation) → Teacher(GPT-4o-mini)의 추론 능력을 Student(Qwen3-14B)에게 이식
학습 후	Judge가 새 사용자의 Profile만 보고 transfer_decisions + Top-10 추천을 직접 생성 가능
왜 QLoRA?	14B 모델을 24~80GB GPU 1대로 학습 가능 (전체 FT 시 80GB+ 필요)
이후 단계	Phase 4 (Ablation 6 conditions) → Phase 5 (Per-Pattern 분석)

2. 학습 데이터 명세

항목	값
파일	data/teacher_train_main.jsonl
줄 수	578줄
포맷	OpenAI/Qwen chat format (system + user + assistant)
한 줄 평균 토큰	약 5,000 토큰 (입력 ~4,500 + 출력 ~500)

P95 토큰	~6,500
최대 토큰	~7,800 (max_seq_length 8192 안)
분할	학습 중 마지막 10% (58줄)를 valid로 자동 분리 (early stopping 트리거)
품질 보장	Codex 외부 리뷰 8개 결함 모두 수정 후 confirmed

train/valid 분리 메커니즘: 본 Phase에서 학습 monitoring용 valid는 train 데이터 마지막 10%를 자동 분리. Phase 4 평가용 test 100명은 학습 데이터에 들어가지 않은 별도 holdout (Phase 2 split 재정의에서 보장).

3. RunPod 환경 셋업

3.1 인스턴스 선택

GPU	VRAM	시간당 요금	적합도
A100 80GB	80GB	~\$1.5-2.0	✅ 권장 — 안전 마진
A100 40GB	40GB	~\$1.0-1.3	가능 (배치=1 필수)
RTX 4090	24GB	~\$0.5-0.8	가능 (max_seq_length 4096까지)
H100 80GB	80GB	~\$2.5-4.0	가능 (속도 ↑ but cost 증가)

3.2 RunPod 템플릿

설정	값
Template	PyTorch 2.2 + CUDA 12.1
Disk	최소 100GB (모델 캐시 + 데이터)
Volume	50GB 권장 (체크포인트 영속화)
Container Start Command	(기본 jupyter 또는 SSH)

3.3 환경 변수

```
HF_TOKEN=hf_새토큰값 # https://huggingface.co/settings/tokens (read 권한)  
WANDB_API_KEY=... # 선택, monitoring용 (https://wandb.ai/settings)
```

4. 학습 실행 — 복붙 가능 명령어

4.1 환경 준비

```
# SSH 접속 후 작업 디렉토리로 이동
cd /workspace # 또는 /root

# 저장소 clone
git clone https://github.com/kkminari/TransferJudge.git
cd TransferJudge

# Python 의존성 설치 (~3-5분)
pip install -r requirements.txt

# HF 로그인 (token은 https://huggingface.co/settings/tokens 에서 발급)
export HF_TOKEN=hf_새토큰값
huggingface-cli login --token $HF_TOKEN

# WandB 로그인 (선택, monitoring용)
export WANDB_API_KEY=...
# 사용 안 할 경우 configs/judge_training.yaml의 report_to를 "none"으로 변경
```

4.2 books_meta 다운로드 (HF Hub)

```
# 약 2.4GB · 5~10분
python3 scripts/download_data_runpod.py --repo kwaksuobusi/transferjudge-data

# 다운로드 확인
ls -lh data/books_meta_filtered.parquet
# 예상: 2.4GB
```

4.3 학습 시작

```
# 백그라운드 실행 + 로그 저장
mkdir -p logs checkpoints

nohup python3 scripts/train_judge.py \
  --config configs/judge_training.yaml \
  > logs/train.log 2>&1 &

# PID 확인
echo "PID: $!"

# 진행 모니터링
tail -f logs/train.log
```

4.4 학습 종료 확인

```
# 종료 감지
grep "학습 완료" logs/train.log
```

```
# 어댑터 크기 확인 (~50MB)
du -sh checkpoints/judge_v1/adapter/

# 최종 메트릭 확인
grep -E "train_loss|eval_loss" logs/train.log | tail -10
```

5. 하이퍼파라미터 설계 + 근거

5.1 LoRA 설정

파라미터	값	근거
r (rank)	16	데이터 578건 (소~중규모) → 16이 적절 (가이드: 500~5,000건 = 16~32, 본 연구는 하한)
lora_alpha	32	$2 \times r$ (스케일링 = $\alpha/r = 2.0$, 표준)
lora_dropout	0.1	소규모 데이터 → 과적합 방지 (SOBA의 0.05보다 강함)
target_modules	q/k/v/o/gate/up/down_proj	Attention + MLP 전체 (Qwen3 표준)
학습 파라미터	~40M / 14B	전체의 ~0.3%만 학습
assistant_only_loss	true	prompt(system/user) 토큰은 loss에서 제외, assistant만 학습 → 추천 JSON 학습 효율 ↑

5.2 학습 hyperparameter

파라미터	값	근거
num_train_epochs	5	소규모 데이터 → 3~5 (early stopping으로 자동 종료 가능)
per_device_batch_size	1	max_seq_length=8192 → VRAM 절약 필수
gradient_accumulation_steps	16	실효 배치 = $1 \times 16 = 16$ (학습 안정성)
learning_rate	$2.0e-4$	QLoRA 표준 (Full FT는 $1e-5$, LoRA는 $1e-4 \sim 5e-4$)
lr_scheduler	cosine	후반부 안정 수렴
warmup_ratio	0.05	전체 스텝의 5% 워밍업
optim	paged_adamw_8bit	메모리 효율적 (QLoRA 표준)
bf16	true	A100/H100 권장 (FP16보다 안정)
gradient_checkpointing	true	메모리 절약 (느리지만 안전)
max_seq_length	8192	P95 토큰 ~6,500 커버 (Phase 2 통계)

5.3 Early Stopping

설정	값
monitor	eval_loss
patience	2 epochs
최종 모델	load_best_model_at_end=true → 최저 eval_loss epoch 사용

resume train_judge.py가 `get_last_checkpoint(output_dir)` 자동 감지 후 `resume_from_checkpoint`로 이어 학습

val split seed=42 shuffle 후 마지막 10% → 분포 치우침 방지

5.4 예상 스텝·시간

항목	계산	값
총 스텝	$578 \times 5 \text{ epochs} \div (1 \times 16)$	약 180 스텝
워밍업	180×0.05	9 스텝
스텝당 시간	A100 80GB 기준 추정	약 2~3분
총 학습 시간	$180 \text{ 스텝} \times 2.5\text{분}$	약 7~8시간

6. 모니터링 + 트러블슈팅

6.1 학습 중 모니터링

지표	정상 범위	이상 신호
train_loss	3.0 → 1.0 (감소)	NaN, ∞, 변화 없음
eval_loss	train_loss + 0~0.5	train보다 크게 높음 → 과적합
학습률	cosine 곡선 추종	0 또는 발산
GPU 사용률	90~99%	50% 이하 → 데이터 IO 병목
VRAM	40~60GB (80GB GPU)	OOM → batch_size 또는 max_seq_length 줄임

6.2 자주 나는 문제 + 해결

증상	원인	해결
OutOfMemoryError	VRAM 부족	(1) per_device_train_batch_size: 1 확인 (2) max_seq_length 를 4096으로 축소 (3) gradient_checkpointing: true 확인
NaN loss	FP 정밀도 문제	(1) bf16: true 인지 확인 (fp16 X) (2) learning_rate 를 5e-5로 낮춤
HF token 오류	인증 실패	huggingface-cli login 재실행. Qwen3-14B는 gated 모델일 수 있음 — 모델 페이지 동의 필요
Connection error	WandB·HF API 일시 장애	(1) report_to: "none" 으로 WandB 비활성화 (2) 30초 대기 후 재시도
학습 너무 느림	I/O 병목	(1) num_workers 증가 (SFTConfig) (2) 데이터셋을 미리 chat template 적용 후 캐시
RunPod 인스턴스 끊김	네트워크	checkpoint가 매 epoch 저장됨 → 재시작 시 resume_from_checkpoint=True

6.3 중단·재개

```
# 중단 (Ctrl+C 또는 SIGTERM)
kill $(pgrep -f train_judge.py)

# 재개 (마지막 checkpoint부터)
python3 scripts/train_judge.py --config configs/judge_training.yaml
# (SFTTrainer가 output_dir의 checkpoint를 자동 감지하여 resume)
```


7. 학습 후 산출물 + 다음 단계

7.1 산출물 디렉토리 구조

```
checkpoints/judge_v1/
├── adapter/                                # ← LoRA 어댑터 (최종 산출, ~50MB)
│   ├── adapter_config.json
│   ├── adapter_model.safetensors
│   ├── tokenizer.json
│   └── tokenizer_config.json
├── checkpoint-36/                          # epoch 1
├── checkpoint-72/                          # epoch 2
├── checkpoint-108/                         # epoch 3 (best)
└── trainer_state.json                     # 학습 메트릭 전체
```

7.2 GitHub 또는 HF Hub로 백업

```
# 옵션 A: GitHub LFS (50MB는 무료 한도 내)
cd checkpoints/judge_v1/adapter
git lfs install
git lfs track "*.safetensors"
git add .gitattributes adapter_model.safetensors adapter_config.json tokenizer*
# (혹은 별도 모델 repo로)

# 옵션 B (권장): HuggingFace Hub
huggingface-cli upload kwaksuobusi/transferjudge-judge-v1 \
  checkpoints/judge_v1/adapter --repo-type=model --private
```

7.3 다음 단계 (Phase 4-5)

Phase	내용	입력	비용·시간
4a	Ablation (c) Ours — 학습된 Judge로 test 100명 추천 평가	checkpoints/judge_v1/adapter + test_users.parquet	\$0.5~\$1, 1-2h
4b	Ablation (a)(b)(d)(f) — 베이스라인 비교	GPT-4o-mini / Claude / w/o Gate / Random	\$2~\$3, 2-3h
4c	Ablation (e) — 전통 CDR 베이스라인	EMCDR/PTUPCDR/TrineCDR 등	\$1, 2-3h
5a	Per-Pattern Ablation (7개 패턴 각각 BLOCK)	Judge × 7 conditions	\$2~\$3, 3-4h
5b	Cold-Start segment 분석	test set의 cold-start cohort 분석	\$0, 1h

7.4 핵심 평가 지표 (Phase 4)

지표	설명	목표
Hit@10	Top-10 안에 GT 책 포함 비율	≥ 60% (Teacher 60.2% 비교 기준)
NDCG@10	순위 가중 평가	높을수록 좋음

MRR	Mean Reciprocal Rank	높을수록 좋음
Pattern Decision Accuracy	transfer_decisions가 Teacher와 일치하는 비율	≥ 80%
JSON 파싱 성공률	유효 JSON 출력 비율	≥ 95%
BLOCK leakage 0건	BLOCK 패턴이 추천 근거로 사용되지 않음	100% 통과

8. 일정 + 비용 요약

단계	작업	예상 시간	예상 비용
0	RunPod 인스턴스 생성 + SSH 접속	10분	-
1	저장소 clone + 의존성 설치	5-10분	~\$0.3 (GPU idle)
2	HF Hub에서 books_meta 다운로드	5-10분	~\$0.3
3	학습 명령 실행 + 진행 모니터링	7-8시간	\$3~\$5
4	어댑터 검증 + HF Hub 업로드	30분	~\$1
Phase 3 총합	-	약 8-9시간	\$5~\$7

한 번에 진행 가능: GPU instance 켜 둔 채 위 명령 순서대로 실행. 학습은 background로 돌리고 다른 일 하면 됨. RunPod 무료 SSH·Jupyter 둘 다 사용 가능.